C Job Scheduler-Full Documentation

By : Segev Kam
Tool Language: C

Note: the documentation assumes you know the basics of C (pointer arithmetics, dynamic allocations, structs, etc.),basic data structures, and atomic operations.

Table Of Contents

Table Of Contents **2**
Introduction **3**
Why bother using it? **4**
Scale and memory usage 4
Context switching 4
The L1 Cache 5
My Solution: the SK Scheduler **6**
The worker thread 6
Workers creation 7
The worker main loop- Initialization 9
The Task Pool 11
The SPSC(Single Producer,Single Consumer) Ring Buffer 16
The SPSC ring buffer design 16
Interaction between the main thread and the workers 19
The Chase-Lev Deque 20
The deque design 20
Memory re-ordering in the Chase-Lev deque 21
Memory orders: relaxed,acquire, release and seq 22
Push,pop and steal implementation 23
Task dependency and virtual locks 25
Parent and child tasks 25
Virtual locks 26
Understanding the main loop 28
The job splitter 32
The splitter's purpose 32
The job splitter function implementation 33
The API functions **35**
Comparing performance- does it actually matter? **36**
Simple array writing 36
Merge sort 38
Concurrent Quick sort 39
Mandelbrot set rendering 40
Conclusion **42**

Introduction

A scheduler,as the name suggests, is a tool that is meant to organize different tasks, events, or jobs and define the order in which they will be done. But when we talk about computer systems, the definition slightly changes.

The most well known scheduler in the world of computer systems is the OS thread scheduler- a key component in the operating system, that defines the order in which different threads are executed at any given time frame(or time slices). It is the backbone of the multi cpu modern systems that allows for many different threads to be executed either simultaneously on different cores or on a single core by utilizing context switches.

My project is a Job scheduler built for and in the C language, that is meant to help developers create jobs to execute and distribute them across worker threads, which is a more optimized, user mode version of the system scheduler.

The scheduler is designed to solve the fundamental scaling problem of multi-core execution: standard OS threads are too heavy for fine-grained tasks, and traditional thread pools suffer from lock contention.

This engine bypasses the OS entirely by implementing a lightweight, lock-free task graph:

- **Almost Zero Thread Overhead:** Spawns a fixed pool of worker threads pinned to each CPU core at boot.
- **Lock-Free Work Stealing:** Uses Chase-Lev deques to allow idle workers to steal work from busy workers without using a single mutex.
- **Dependency Graph:** Tasks can spawn sub-tasks and wait for them to finish using virtual atomic locks, completely eliminating thread-blocking.

Each part of the scheduler will be expanded on in its own dedicated section.

Why bother using it?

As I said before, the scheduler already exists inside the OS, so why even bother using a custom scheduler to complete a job?

For many simple jobs, normal multi-threaded task execution, where each task is assigned its own thread and executed is enough to get the job done,but as we scale and optimize, we find that this solution comes with a bundle of problems.

Scale and memory usage

At first glance, spawning a new OS thread for every job doesn't seem like a big deal. If you want to process a big chunk of data, why not just spawn 100 threads to complete 100 tasks?

The issue becomes obvious when you look at what an OS thread actually costs. Every single thread holds an execution context (state registers, thread ID, priority, etc.) taking up kernel memory. Even worse, the OS reserves a massive chunk of virtual address space for every thread's stack–usually 1MB on Windows and 8MB on Linux.

If you spawn 100 OS threads to handle 100 tasks, you are eating up roughly 800MB of virtual memory space, even just to execute a few basic functions.

Context switching

It's quite ironic to include one of the mechanics that allows multi-threading to even be possible as one of its downsides in job scheduling, but there is a big reason for it.
Context switching between threads is not a free operation that magically switches which threads the cpu executes, and it is certainly a costly one. Every time the OS does a context switch, it needs to do multiple things before continuing:

1. It needs to stop the current thread executing in the cpu core
2. It needs to save the current execution context of the thread running(the current IP for example) inside the kernel memory segment of the thread(specifically on the process' kernel stack).
3. It needs to load the cpu state of the thread it needs to context switch to into the cpu.
4. Only then can it continue to execute the new thread.

On a small scale, this operation is really minuscule. But let's take the example from before and say we allocate a thread for each task. If we could assume the priority of all tasks is around the same and that we are running on a 6 core computer(which in modern times can run 12 threads simultaneously), 100 tasks can't run all at the same time, we would have to context switch at some point to allow each task to run roughly the same. In a perfect situation with zero interruption from external threads , every core would need to perform hundreds or even thousands of context switches a second just to give every thread a fair slice of time. Instead of executing the actual logic, the CPU ends up wasting too many clock cycles just acting as a traffic cop, constantly saving and loading states. The overhead from the context switching can become quite noticeable.

The L1 Cache

The L1 cache is a small and really fast memory built on top of each cpu core, usually around the size of 32-128 KB. Because each core has its own cache, when data is loaded from memory into a specific L1 cache, every other L1 cache won't have that data.

The term cache miss is a known annoyance when we try to optimize and every nanosecond matters. It happens when the cpu tries to access a specific address in memory but realizes it's not in the L1 cache, it needs to bring it over from another cache(like L2) or in the worst case- the ram, into its own L1 cache before it actually accesses the data in that memory location. This operation is also really costly when we try to maximize the optimization.

The reason this is very important is that when the operating system pauses a thread and wakes it up on a completely different CPU core, that thread leaves all of its hot L1 data behind. The new core is forced to suffer a massive wave of cache misses just to resume execution, pulling all that data from main RAM all over again.

So lets use once again the same example of 100 threads for 100 tasks. If all the tasks need to read the same really big data array at different sections of it, when a thread that works from index i to index k is switched out and wakes up again in a different core, it will be forced to grab all that data again into that new core. Now imagine that happening thousands of times a second for all cores between all threads. The overhead of that will be incredibly noticeable.

For those main reasons and many more like locking and False sharing that all need to be handled by the developer, we have to minimize the amount of threads we allocate for completing the job and making sure they each stay off of each other's ways as much as possible.

My Solution: the SK Scheduler

This project was built to solve the scaling and memory problems mentioned above, while strictly avoiding the use of locks to prevent thread starvation and maximize CPU efficiency. In the following sections, I will break down each core component of the scheduler and explain exactly how it works under the hood.

Before starting anything, I'll make a quick differentiation between a job and a task:

- A job is the entire work that you want to be executed and completed. It can range from updating every entity in a game, or as simple as summing an integer array.
- A task is a chunk of that job that has the same end goal, yet isn't meant to complete the entire job. A job is split into tasks and only when all tasks are complete will the job be marked as completed. An example could be a task that is summing only the first quarter of the array.

The worker thread

Instead of allocating a thread for each individual task, we need to optimize how we spawn the threads. For that we need to ask the question- what is the maximum amount of workers we can spawn to maximize task consumption and execution, while also minimizing context switching and memory bloat?

My solution for this dilemma- 1 worker for each individual logical hardware thread (2 workers per 1 physical core). The reason for it is that since we want each worker to work on a separate core without moving to a new one mid execution, we could pin each worker to a different core. That way each worker will always have his own "private" L1 cache, and the workers will never need to context switch and fight for cpu time between themselves.

The implementation is as followed:

First, we have to define our scheduler worker struct:

```
typedef struct{
    alignas(64) size_t core_id;

    sk_thread_t thread_handler;

    atomic_bool termination_flag;
    sc_cl_deque cl_deque;

    alignas(64) atomic_size_t rb_head;
    size_t rb_tail_cache;

    alignas(64) atomic_size_t rb_tail;
```

```
    size_t rb_head_cache;

    size_t rb_capacity_mask;
    sc_task** rb_buffer;

}sc_worker;
```

I will focus only on the first three fields as they are the ones that matter:

- The core_id is the core in which this worker is strictly running on.
- The thread_handler is the actual thread information that is used by the OS.
- The termination_flag is a flag that signals the worker thread when he needs to exit and his work has finished.

Workers creation

To figure out how many cores the device the scheduler is running on has, we need to create an OS specific function that queries the OS to know how many cores it has:

```
size_t get_cpu_logical_proccessors(void) {
#if defined(_WIN32)
    SYSTEM_INFO sysinfo;
    GetSystemInfo(&sysinfo);
    return sysinfo.dwNumberOfProcessors;
#elif defined(_SC_NPROCESSORS_ONLN)
    return sysconf(_SC_NPROCESSORS_ONLN);
#elif defined(_SC_NPROCESSORS_CONF)
    return sysconf(_SC_NPROCESSORS_CONF);
#else
    return 1; // Fallback default if unknown
#endif
}
```

With that function, we can now know how many worker we want to allocate inside the scheduler boot:

```
worker_pool=allocate_worker_pool(lp_count,spsc_queue_size,cl_deque_size);
    if(!worker_pool){
        fprintf(stderr, "Fatal Hardware Error: Failed to allocate worker
pool.\n");
        return;
    }
```

```
    for(size_t i=0;i<lp_count;i++){
        #if defined(_WIN32)
            worker_pool[i].thread_handler = CreateThread(NULL, 0,
(LPTHREAD_START_ROUTINE)worker_main_loop, &worker_pool[i], 0, NULL);
            if(!worker_pool[i].thread_handler){
                fprintf(stderr,"Thread Creating Error:Thread #%lu couldn't
be created",i+1);
            }
        #else
            int result=pthread_create(&worker_pool[i].thread_handler,
NULL, worker_main_loop, &worker_pool[i]);
            if(result!=0){
                fprintf(stderr,"Thread Creating Error:Thread #%lu couldn't
be created",i+1);
            }
        #endif
    }
```

The worker main loop- Initialization

Each worker starts by pinning itself to the core it is meant to run on,and from there it has its own loop it runs infinitely until the termination flag is set to true by an external thread:

```
static inline void pin_thread_to_core(size_t core_id) {
	#if defined(__linux__)
	    cpu_set_t cpuset;
	    CPU_ZERO(&cpuset);
	    CPU_SET(core_id, &cpuset);
	    pthread_t current_thread = pthread_self();
	    if (pthread_setaffinity_np(current_thread, sizeof(cpu_set_t),
	&cpuset) != 0) {
	        fprintf(stderr, "Hardware Warning: Failed to pin thread to
	core %zu\n", core_id);
	    }
	#elif defined(_WIN32)
	    HANDLE thread = GetCurrentThread();
	    // Bitwise shift to generate the affinity mask for the target
	core
	    DWORD_PTR mask = (DWORD_PTR)1 << core_id;
	    if (SetThreadAffinityMask(thread, mask) == 0) {
	        fprintf(stderr, "Hardware Warning: Failed to pin thread to
	core %zu\n", core_id);
	    }
	#elif defined(__APPLE__)
	    // Darwin/XNU kernel does not permit strict pinning, but
	supports "Affinity Tags".
	    // This hints the scheduler to keep threads sharing a tag on the
	same L2/L3 cache cluster.
	    thread_port_t mach_thread =
	pthread_mach_thread_np(pthread_self());
	    thread_affinity_policy_data_t policy = { (integer_t)core_id };
	    thread_policy_set(mach_thread, THREAD_AFFINITY_POLICY,
	(thread_policy_t)&policy, 1);
	#else
	    // Fallback for unsupported architectures
	    fprintf(stderr, "Hardware Warning: Thread pinning not supported
	on this OS.\n");
	#endif
```

```
}
void* worker_main_loop(void* arg) {
    sc_worker* local_worker = (sc_worker*)arg;
    tl_current_worker = local_worker;
    uint32_t rng_state = (uint32_t)(local_worker->core_id + 1) *
2654435761U;
    size_t total_cores = get_cpu_logical_proccessors();
    if (total_cores < 1){
        total_cores = 1;
        printf("Couldn’t query cpu cores amount\n");
    }
    pin_thread_to_core(local_worker->core_id);
    while (!atomic_load_explicit(&local_worker->termination_flag,
memory_order_relaxed)) {
      ...
      ...

}
return NULL;
```

I will explain what's inside the while loop once all the other concepts will be explained.

The Task Pool

Before we start discussing about the pool, let's take a look at the task struct:

```
//the function each worker will call to execute( written and passed by the
developer)
typedef void (*sc_task_fn)(void* user_data, size_t start_idx, size_t
end_idx);
typedef struct sc_task_i{
   alignas(64) void* task_payload;
    size_t start_inx;
    size_t end_inx;
    sc_task_fn task_fn;
    //count of children tasks still running(while this task is waiting)
    atomic_size_t unfinished_dependencies;
    //which parent task needs this task completed, null if no one
    struct sc_task_i* parent_task;
    //index for the free list in the memory pool
    _Atomic uint32_t next_free_idx;
}sc_task;
```

Every task has 7 fields:

1. The payload- user data given to the task. It can be a string, a pointer to an int, an array, basically every type the user needs for his job. Because the scheduler never touches the data inside the payload pointer, it remains a void* and casted inside the task function by the developer.
2. Start index and end index- basically implies which chunk of the entire payload the function is processing.
3. the task function- the function the worker will call when processing the task
4. Unfinished dependencies- will be expanded upon later
5. Parent task- a pointer to the parent task who created this task
6. Next free index- will be explained when discussing about the memory pool

Another problem that rises when every nanosecond counts and we want to avoid locking at all cost to prevent thread starvation, is allocating tasks on the fly.

The main issue with malloc is that it has a locking mechanism to make it thread safe. Meaning if two threads try to allocate a new task on the fly at the same time, even though it will be safe, one will have to wait for the other to release the lock for it to continue. So we cannot use malloc, so what can we do instead?

That's where the memory pool comes into play.

The memory pool is a pre-allocated pool of tasks with a pre-defined size that is allocated at boot time. After the scheduler is booted, before we start any job, we will have all the tasks already allocated.

The global task array is defined in the data segment as followed:

```
// global memory pool,allocated at boot
alignas(64) atomic_uint_least64_t global_pool_head;
alignas(64) sc_task* global_task_array = NULL;
size_t global_task_capacity = 0;

//every thread has his own cache of tasks
_Thread_local sc_task* task_cache[TASK_CACHE_CAPACITY];
_Thread_local size_t task_cache_count = 0;
```

As you can see I also added a thread local task cache, to reduce how much the workers will have to access the global array.

The memory pool is designed so that every task has an index to the next free task in the memory pool(hence why the next_free_idx field in the task struct).

When we acquire from the memory pool, the head becomes the next free index of the current head task.
When we release a task to the memory pool, the head becomes the new task released, and the next free index of the new released task points to the old head task.

The main problem with using a memory pool for the only source of task generation across all worker threads, is a lock free problem known as the "ABA problem".
The ABA problem is a an issue with the CAS(compare and swap) atomic operation, which is used to swap the head of the memory pool.

CAS is an atomic operation that receives 3 values- first is the address of an atomic current value, the address of an expected value, and the third is a new value.
If the expected value matches the current value, we load the new value to the current value.
If its not matching, the expected value becomes the value in the current value(which is why we also pass its address to the CAS operation).

Let's look at an example of the ABA problem inside our task memory pool:
The memory pool looks like this:**Head -> Task A -> Task B -> Task C**

Thread 1 wants to claim a task. It reads head (Task A) and it reads the next item (Task B).
Suddenly, the OS context-switches Thread 1 and puts it to sleep.
While Thread 1 is asleep, Thread 2 wakes up and goes extremely fast.

- It pops Task A and uses it.
- It pops Task B and uses it. (The Head is now Task C)
- Thread 2 finishes its job with Task A and pushes Task A back onto the pool.

The pool now looks like this: **Head -> Task A -> Task C** (Notice that task B is completely gone–it is currently being used by Thread 2)

Thread 1 wakes up. It resumes exactly where it left off at Step 3: the CAS instruction. It executes: "If the Head is still Task A, change the Head to Task B."
Thread 1 looks at the pool and sees the head is still task A. Thread 1 has no idea that Task A was popped and put back. The CAS instruction succeeds.

Now the new head becomes task B, which is still used by thread 2, thus creating a memory corruption.

To avoid this, we use a solution to the problem known as tagged pointers(or tagged indexes in our case). The head index is composed of 64 bits, where 32 bits are for the index itself in the array, and the other 32 bits are for what is known as the tag. The tag is an indicator- every time we call acquire or release, the tag is increased by 1. Now when the CAS looks at the head and tries to swap it with the next head, the value it compares is not only the index, but the tag as well. Meaning if the head was changed between the thread packing the expected head and the CAS, the CAS will fail, and the ABA problem is mitigated.

Here is the implementation:

```
#define POOL_EMPTY_INDEX 0xFFFFFFFF
static inline uint64_t pack_aba(uint32_t index, uint32_t tag) {
    return ((uint64_t)tag << 32) | index;
}
static inline uint32_t unpack_index(uint64_t packed_val) {
    return (uint32_t)(packed_val & 0xFFFFFFFF);
}
static inline uint32_t unpack_tag(uint64_t packed_val) {
    return (uint32_t)(packed_val >> 32);
}
sc_task* memory_pool_acquire() {
    //if thread cached memory pool contains any available tasks, pop it
from there
    if(task_cache_count>0){
        return task_cache[--task_cache_count];
    }
    //if cache is empty, grab a task from global memory pool
    uint64_t current_head = atomic_load_explicit(&global_pool_head,
memory_order_acquire);
    uint32_t head_idx;
    while (true) {
```

```
        head_idx = unpack_index(current_head);
        if (head_idx == POOL_EMPTY_INDEX) {
            return NULL;
        }
        uint32_t next_idx =
atomic_load_explicit(&global_task_array[head_idx].next_free_idx,
memory_order_relaxed);
        uint32_t current_tag = unpack_tag(current_head);
        uint64_t new_head = pack_aba(next_idx, current_tag + 1);
        if (atomic_compare_exchange_weak_explicit(
                &global_pool_head,
                &current_head,
                new_head,
                memory_order_acquire,
                memory_order_acquire)) {
            break;
        }
        // force the pipeline to pause and back off the memory bus
        cpu_relax();
    }
    return &global_task_array[head_idx];
}
void memory_pool_release(sc_task* task) {
    if(task_cache_count < TASK_CACHE_CAPACITY){
        task_cache[task_cache_count++] = task;
        return;
    }
    // cache full, evict(is it evict or avict) half the task cache
    const size_t start_evict = TASK_CACHE_CAPACITY - TASK_EVICT_COUNT;
    for (size_t i = start_evict + 1; i < TASK_CACHE_CAPACITY; i++) {
        uint32_t prev_idx = (uint32_t)(task_cache[i - 1] -
global_task_array);
        atomic_store_explicit(&task_cache[i]->next_free_idx, prev_idx,
memory_order_relaxed);
    }
    uint32_t current_task_idx = (uint32_t)(task - global_task_array);
    atomic_store_explicit(
        &task->next_free_idx,
        (uint32_t)(task_cache[TASK_CACHE_CAPACITY - 1] -
global_task_array), memory_order_relaxed );
```

```
    uint32_t tail_idx = (uint32_t)(task_cache[start_evict] -
global_task_array);
    uint64_t current_head = atomic_load_explicit(&global_pool_head,
memory_order_relaxed);
    while (true) {
        uint32_t old_head_idx = unpack_index(current_head);
        global_task_array[tail_idx].next_free_idx = old_head_idx;
        uint32_t current_tag = unpack_tag(current_head);
        uint64_t new_head = pack_aba(current_task_idx, current_tag + 1);
        if (atomic_compare_exchange_weak_explicit(&global_pool_head,
&current_head, new_head,memory_order_release, memory_order_relaxed)) {
            break;
        }
        cpu_relax();
    }
    task_cache_count = start_evict;
}
```

The SPSC(Single Producer,Single Consumer) Ring Buffer

The SPSC ring buffer design

Now that you know how tasks are allocated, let's talk about sharing the initial jobs with the workers.

The scheduler needs a strict, highly efficient boundary between the main thread (which creates the jobs) and the Worker Threads (which execute them). If the main thread had to acquire a mutex lock every time it submitted a job to the engine, it would completely bottleneck the application's main loop.

To solve this, you pass jobs through a lock-free SPSC Ring Buffer. This acts as a fast, one way bridge into the lock-free environment:

- **Single Producer:** Only the main thread is permitted to push new jobs into the buffer.
- **Single Consumer:** Only one designated worker (or dispatcher) is permitted to pop jobs out of the buffer.

this buffer is a fixed-size array connected end-to-end in a circle, tracked by two atomic integers: a head and a tail.

When the main thread submits a job, it writes to the tail index and increments it. When the scheduler pulls a job, it reads from the head index and increments it. As long as the Head doesn't catch up to the Tail (meaning the buffer is empty) and the Tail doesn't lap the Head (meaning the buffer is full), both threads operate simultaneously in complete isolation.

To maximize hardware performance, the engine uses bitwise AND masking (capacity-1) instead of the much slower modulo operator to safely wrap the indices around the ring, ensuring job submission executes in just a few nanoseconds.

Here is the structure implementation:

```
typedef struct {
    // external thread exclusive cache line
    alignas(64) atomic_size_t tail;
    size_t head_cache; // local cache to prevent true sharing memory reads
     // wroker exclusive cache line
    alignas(64) atomic_size_t head;
    size_t tail_cache;
    size_t capacity_mask; // capacity - 1 for bitwise wrapping
    sc_task** buffer;
} sc_spsc_queue;
```

Now it's an appropriate time to discuss a concept that you have seen pop up multiple times in the code already, which is the cache line.

When a modern CPU reads data from the RAM into its L1 cache, it doesn't fetch that data one byte at a time. Instead, it fetches memory in fixed-size blocks called cache lines–which are almost always 64 bytes long on modern processors.

If a CPU core needs to read a standard 4-byte integer, it grabs that integer along with the 60 bytes of memory next to it, pulling the entire 64-byte chunk into the L1 cache. In single-threaded programming, this is a great hardware optimization. It means nearby variables are pre-loaded for free.

But in multi-threaded, lock-free programming, this creates a hidden performance penalty called false sharing.

Imagine two separate atomic variables, like the Ring Buffer's head and tail. They do two completely different things, but the compiler naturally packs them side-by-side in memory to save space. Because they are next to each other, they end up sharing the exact same 64-byte cache line.

Imagine how the CPU hardware handles this:

- The main thread is constantly modifying the tail.
- The worker thread is constantly modifying the head.

Every single time Core 1 updates the tail, the CPU hardware assumes the entire 64-byte cache line is now dirty and outdated. To keep memory safe, the CPU sends a signal across the motherboard to instantly invalidate that cache line.

A microsecond later, the worker thread tries to read its head variable, but suffers a cache miss because Core 1 just wiped that cache line. The CPU is forced to stall the thread and fetch the data from main RAM all over again. The cores end up constantly destroying each other's L1 cache, even though they aren't actually sharing variables.

This is why you see the alignas(64) keyword used throughout the scheduler's structs. By forcing independent atomic variables to align to 64-byte boundaries, we guarantee the compiler places them on completely separate cache lines. The main and the worker threads can now manipulate the SPSC queue in isolation, completely eliminating false sharing.

There is still one problem left. Every time we pop we need to check if the head and tail meet(to see if there are tasks left), and every time we push we need to do the same as well(to check if the spsc buffer isn't full). Meaning if we only have the head and the tail, both the main thread and the worker thread will hold both the head cache line and the tail cache line, basically invalidating what we try to optimize.

For that, we introduce the cached head and tail. The cached head and tail are only read by their respective threads,until it is completely necessary to read the actual tail and head by the

other threads. That means that until then both threads will grab a single cache line that contains either head and cached_tail, or tail and cached_head, which will reduce false sharing by a lot.

The push and pop operations:

```
static inline sc_task* spsc_consumer_pop(sc_worker* worker) {
    size_t current_head = atomic_load_explicit(&worker->rb_head,
memory_order_relaxed);
    if (current_head >= worker->rb_tail_cache) {
        worker->rb_tail_cache = atomic_load_explicit(&worker->rb_tail,
memory_order_acquire);
        if (current_head >= worker->rb_tail_cache) {
            return NULL;
        }
    }
    sc_task* task = worker->rb_buffer[current_head &
worker->rb_capacity_mask];
    atomic_store_explicit(&worker->rb_head, current_head + 1,
memory_order_release);
    return task;
}
static inline bool spsc_producer_push(sc_worker* worker, sc_task* task) {
    size_t current_tail = atomic_load_explicit(&worker->rb_tail,
memory_order_relaxed);
    size_t capacity = worker->rb_capacity_mask + 1;
    if (current_tail - worker->rb_head_cache >= capacity) {
        worker->rb_head_cache = atomic_load_explicit(&worker->rb_head,
memory_order_acquire);
        if (current_tail - worker->rb_head_cache >= capacity) {
            return false;
        }
    }
    worker->rb_buffer[current_tail & worker->rb_capacity_mask] = task;
    atomic_store_explicit(&worker->rb_tail, current_tail + 1,
memory_order_release);
    return true;
}
```

Interaction between the main thread and the workers

Now that each worker has its own spsc buffer, we need a way for the main thread to choose which spsc buffer it will push to.

Since workers steal work from each other, I didn't find any valid reason to use any sort of priority, so I used a simple load balancing strategy called Round-Robin.

Round-robin is a system where people, tasks, or resources take turns in an equal circular order so that everyone gets an equal share.

In our use case, when the program submits a job, the scheduler will choose a different worker to push the job to its spsc queue.

Here's how it's implemented:

```
//robin round counter for worker
static atomic_size_t rr_worker_index = ATOMIC_VAR_INIT(0);
bool scheduler_submit_job(sc_job* job,size_t chunk_size){
      ...
      ...
      size_t worker_count = get_cpu_logical_proccessors();
     if (worker_count < 1) worker_count = 1;
    size_t target_idx = atomic_fetch_add_explicit(&rr_worker_index, 1,
memory_order_relaxed) % worker_count;
    bool success =spsc_producer_push(&worker_pool[target_idx],
child_task);
    if (!success) {
        //if spsc if full, return the task to the pool
        memory_pool_release(root_task);
        return false;
    }
  return true;
}
```

You can see an example of when we release a task as well as how we choose to push it to the worker.

The Chase-Lev Deque

The deque design

After we figured out how the external thread passes jobs to the workers, we can move on to arguably the most important part of the scheduler: how the workers execute those tasks and dynamically balance the workload among themselves.

In a perfect world, if we split a massive job into 4 separate chunks for 4 CPU cores, every core would finish at the exact same millisecond. But in reality, some tasks take longer to process, and the OS might interrupt one of our cores. If Core 1 finishes early, we do not want it sitting idle while Core 2 does all the heavy lifting.

To solve this, we use a work-stealing architecture. Every worker thread owns a specialized lock-free data structure called a Chase-Lev Deque (Double-Ended Queue).

Here is how the workers interact with it:

1. The Owner (LIFO / Stack Behavior) When a worker spawns tasks or sub-tasks, it pushes them to the bottom of its own deque. When the worker needs a new task to execute, it pops from the bottom.

Why? Because treating the deque like a stack guarantees cache locality. The task that was just initialized is still hot in the CPU's L1 cache, so the worker will always be able to fetch it from its L1 cache.

2. The Thief (FIFO / Queue Behavior) If a worker's deque becomes completely empty, it becomes a thief. It will randomly select another worker and try to steal a task from the top of their deque.

Because the owner and the thief are operating on opposite ends of the deque, they almost never contend for the same memory,and when they finally do (when only one task is left), the deque resolves it safely using the atomic CAS operation which ensures a task is never executed twice, and completely eliminating the need for a mutex.

Theres also another huge benefit that comes because the thief steals from the top.In divide and conquer algorithms, the oldest tasks at the top of the deque are usually the largest, least divided chunks of work. By stealing from the top, the thief steals a massive chunk of work for itself, minimizing the number of times it needs to steal again, while completely avoiding the owner who is busy working at the bottom.

The struct is implemented as followed:

```
typedef struct{
    alignas(64) atomic_int_least64_t top;
    alignas(64) atomic_int_least64_t bottom;
```

```
   //since top and bottom only increments, the top cache will always
be an overestimation
    //of how full the deque is, which is useful for pushing when we
dont want to always load the actual top
    int64_t top_cache;
    alignas(64) sc_task** buffer;
    int64_t capacity_mask;
}sc_cl_deque;
```

As you can see, it uses the same cached index trick as the SPSC.

Memory re-ordering in the Chase-Lev deque

As I just mentioned, when there is only one task left, the atomic CAS operation is used to make sure only one worker can take on a task at a time.

In modern times, the multi-core hardware requires us to solve a big issue with that approach known as memory re-ordering.

When you write code, you assume the computer executes your instructions exactly in the order you typed them. But in reality, your compiler and your CPU's hardware are rearranging your instructions behind the scenes to maximize speed (this is called Out-of-Order Execution).

In a single-threaded program, this is fine–the CPU guarantees the final result will be the same. But in a lock-free multithreaded engine, this reordering can cause horrible issues.

Imagine the owner thread is pushing a new task to the bottom of its deque. Logically, it happens in two steps:

1. Write the task pointer into the array.
2. Increment the bottom index so others know the task is there.

Now, imagine the CPU decides to reorder these two instructions to save a few nanoseconds. It increments the bottom index first.

At that exact nanosecond, a thief thread sees the bottom index has increased, so it assumes there is a valid task to steal. The thief grabs the memory from the array and tries to execute it–but because of the reordering, the owner hasn't actually written the task pointer yet. The thief executes garbage memory and most likely crashes the program.

This all works under the hood with the load and store buffers inside the CPU. I won't explain about them here, but you are more than welcome to read about them yourself.

Memory orders: relaxed,acquire, release and seq

To solve this, as you may have already seen with some of the atomic instructions before, C as well as most other languages have introduced memory orders inside the atomic operations.

There are 4 main types, each with a different purpose:

1. Relaxed (memory_order_relaxed)

- What it does: it provides no ordering guarantees. The CPU and compiler are completely free to reorder instructions around it. The only thing it guarantees is that the operation itself is atomic..
- When we use it: we use this when we don't care about the sequence. For example, if a worker is just incrementing a statistical counter, or updating an isolated index where no other memory relies on it. Because it doesn't restrict the CPU, it is the fastest memory order.

2. Release (memory_order_release)

- What it does: used when writing to a variable. It acts as a one-way hardware fence. It tells the CPU that every memory write that happened before this line of code must be fully committed and visible before you execute this atomic write.
- When we use it: used by the owner thread when pushing a task. It ensures the task's data is fully written into the deque's array before the bottom index is incremented.

3. Acquire (memory_order_acquire)

- What it does: used when reading a variable. It acts as a one-way fence in the opposite direction. It tells the CPU it is not allowed to execute any subsequent reads or writes until this atomic read fully completes.
- When we use it: used by the thief thread when stealing. It ensures the thief fully reads the updated bottom index before it tries to access the actual task pointer inside the array. When paired together, acquire and release form a safe bridge between two different CPU cores.

4. Sequentially Consistent (memory_order_seq_cst)

- What it does: this is the last option we want to choose. It places a two-way hardware fence that stops all instruction reordering in both directions and enforces a strict, global chronological order across all CPU cores.
- When we use it: This is the default memory order in C11 if you don't explicitly specify one. However, because it forces the CPU to stall its execution pipeline and synchronize with every other core on the motherboard, it is incredibly slow. In a high performance engine like mine, I intentionally avoided this, relying on the much faster acquire release pairs to get the synchronization we need without the performance penalty.

Push,pop and steal implementation

```
static inline bool chase_lev_push(sc_cl_deque *deque, sc_task *task){
    int64_t b=atomic_load_explicit(&deque->bottom, memory_order_relaxed);
    int64_t capacity = deque->capacity_mask+1;
    //since t and b only increments, the top cache will always be an
overestimation
    //of how full the deque is
    if(b-deque->top_cache>=capacity){
        deque->top_cache= atomic_load_explicit(&deque->top,
memory_order_relaxed);
        if (b - deque->top_cache >= capacity) return false;
    }
    deque->buffer[b & deque->capacity_mask]=task;
    atomic_store_explicit(&deque->bottom, b + 1, memory_order_release);
    return true;
}
static inline sc_task* chase_lev_pop(sc_cl_deque* deque) {
    int64_t b = atomic_load_explicit(&deque->bottom,
memory_order_relaxed);
    int64_t t = atomic_load_explicit(&deque->top, memory_order_relaxed);
    // if deque already empty(since its his deque we dont need to check
further since noone else can push)
    if (b <= t) {
        return NULL;
    }
    //decrememnt bottom
    b--;
    atomic_store_explicit(&deque->bottom, b, memory_order_relaxed);
    atomic_thread_fence(memory_order_seq_cst);
    t = atomic_load_explicit(&deque->top, memory_order_relaxed);
    int64_t size = b - t;
    if (size < 0) {
        atomic_store_explicit(&deque->bottom, t, memory_order_relaxed);
        return NULL;
    }
    sc_task* task = deque->buffer[b & deque->capacity_mask];
```

```
    if (size > 0) return task;
    int64_t expected_t = t;
    // attempt CAS race against any thieves
    bool won_race = atomic_compare_exchange_strong_explicit(
        &deque->top,
        &expected_t,
        t + 1,
        memory_order_seq_cst,
        memory_order_relaxed
    );
    //if CAS failed, thief got the item first
    if (!won_race) task=NULL;
    atomic_store_explicit(&deque->bottom, t + 1, memory_order_relaxed);
    return task;
}
static inline sc_task* chase_lev_steal(sc_cl_deque *victim_deque){
    int64_t t = atomic_load_explicit(&victim_deque->top,
memory_order_acquire);
    atomic_thread_fence(memory_order_seq_cst);
    int64_t b = atomic_load_explicit(&victim_deque->bottom,
memory_order_acquire);
    if (b <= t) {
        return NULL;
    }
    sc_task* task= victim_deque->buffer[t & victim_deque->capacity_mask];
    //if CAS faield, thief got the item first
    if (!atomic_compare_exchange_strong_explicit(
        &victim_deque->top,
        &t,
        t + 1,
        memory_order_seq_cst,
        memory_order_relaxed)) return NULL;
    return task;
}
```

Task dependency and virtual locks

Parent and child tasks

The last piece we have yet to discuss are sub-tasks.

In many cases, a job requires only one function executing for each chuck, but sometimes it's multiple.

Take for example a simple recursive merge sort. If we want to use it with our task scheduler, we would need that each task would split itself to two sub tasks- left and right recursively until a certain depth.

But because it's not actually a single-threaded recursive algorithm, the parent isn't supposed to wait in a loop for both its left and right task to finish until he continues, since that would break the entire multi-threaded solution.

So to fix that, we introduce task dependency.

Each task has 2 fields:

```
//count of children tasks still running(while this task is waiting)
atomic_size_t unfinished_dependencies;

//which parent task needs this task completed, null if no one
struct sc_task_i* parent_task;
```

The unfinished dependencies are the amount of sub tasks that are currently still not finished that this task has created.

The parent task is a pointer for children tasks to point to their parent task.

Before spawning any sub task, it sets its own function(the task_fn field) to what it will continue executing after all sub tasks finish.After that,when the parent task spawns a sub task in any way, it increases its unfinished dependencies to the amount of sub tasks it has created, and set their parent task to itself.
Once it has finished creating all sub tasks, it parks itself in memory as it remains the parent pointer the sub tasks point to, meaning he isn't pushed again to the deque.
At the end, after all sub tasks finish, the last sub task sees that there are no more sub tasks remaining, and pushes the parent back into the deque, to continue executing what it has set itself to do before spawning the sub tasks.

Virtual locks

While this design works, it exposes a massive problem, which is called premature execution.

Sorting an array is a great example of Data Parallelism (running the same function on different chunks of data). But real world applications usually require Task Parallelism–executing completely different functions simultaneously.

A good example could be a backend to an online shopping service. When a user tries to pay, it will need to:

1. Verify the credit card
2. Query the database for the items
3. Calculate a fast shipping route to his home

If we would want to create a task that spawns those 3 sub tasks to manage the payment simultaneously, our dependency strategy could suffer majorly.

If the credit card verification task is pushed to the deque, stolen by another worker, and finished before the other 2 sub tasks are even pushed, we would have a major issue where the first task thinks the unfinished dependencies is 0(since the other two weren't pushed yet), and push the parent task back before the whole process has been completed.

For that reason, we need to condition the children to wait for the parent to first finish executing its initial task, and only after that can the parent be pushed back when ALL sub tasks finish.

To overcome this without introducing any locks that would starve other worker threads, I used a technique called "virtual locking".

With virtual locks, every time a task is spawned, no matter if it will spawn a sub-task or not, its unfinished dependencies field is already set to 1. The reason is that we consider the parent task as its own sub-task condition(if that makes sense), and when the worker finishes a task, it will always decrease that task's dependencies by 1.

That way we condition both the parent and the children- whoever of ALL tasks, both the parent or children finishes last, has to push the parent back to the deque.

Let's look at the example from before:

The main checkout task begins execution. Its dependencies counter is immediately set to 1(representing the parent itself).

The parent spawns the verify card task. It increments the dependency counter to 2, and pushes the task into the deque.

A thief worker on another core immediately steals the payment task, executes it in a microsecond, and finishes. It decrements the dependency counter back down to `1`. But now the counter hasn't hit `0`, so it won't push the parent back.

Now when the parent task is done, it decrements its own counter by one, which drops the virtual lock. So no matter if he is the last finishing or one of the children is, it is guaranteed that no premature execution could happen.

Understanding the main loop

The full worker main loop is implementation is as follows:

```
void* worker_main_loop(void* arg) {
    sc_worker* local_worker = (sc_worker*)arg;
    tl_current_worker = local_worker;
    uint32_t rng_state = (uint32_t)(local_worker->core_id + 1) *
2654435761U;
    size_t total_cores = global_worker_count;
    if (total_cores < 1){
        total_cores = 1;
        fprintf(stderr, "Warning: Failed to query logical processor count,
defaulting to 1.\n");
    }
    pin_thread_to_core(local_worker->core_id);
    while (!atomic_load_explicit(&local_worker->termination_flag,
memory_order_relaxed)) {
        tl_current_task = NULL;
        tl_current_task = chase_lev_pop(&local_worker->cl_deque);
        // if chase lev queue is empty, try finding a task in the spsc
queue
        if (!tl_current_task) {
            tl_current_task = spsc_consumer_pop(local_worker);
        }
        //steal work
        if (!tl_current_task) {
            if (!tl_current_task && total_cores > 1) {
                size_t victim_id = xorshift32(&rng_state) % (total_cores -
1);
                if (victim_id >= local_worker->core_id) {
                    victim_id++; // Uniformly maps [0 .. N-2] to all cores
except local_worker->core_id!
                }
                tl_current_task =
chase_lev_steal(&worker_pool[victim_id].cl_deque);
            }
        }
        if (!tl_current_task) {
            cpu_relax();
            continue;
```

```
        }
        sc_task_fn current_fn = tl_current_task->task_fn;
        tl_current_task->task_fn = NULL;
        // execute the function
        current_fn(tl_current_task->task_payload,
tl_current_task->start_inx, tl_current_task->end_inx);
        //drop the virtual lock
        size_t remaining_deps = atomic_fetch_sub_explicit(
            &tl_current_task->unfinished_dependencies,
            1,
            memory_order_acq_rel
        );
        // the loop allows chaining of continuation and spawn sub task
        //meaning continuation function can also spawn sub tasks infinitly
without any problems
        if (remaining_deps == 1) {
            // all children spawned by current_fn already finished while
current_fn was running
            // If current_fn registered a continuation, run it right now
inline.
            while (tl_current_task->task_fn != NULL) {
                sc_task_fn continuation_fn = tl_current_task->task_fn;
                tl_current_task->task_fn = NULL;
atomic_store_explicit(&tl_current_task->unfinished_dependencies, 1,
memory_order_relaxed);

                continuation_fn(tl_current_task->task_payload,
tl_current_task->start_inx, tl_current_task->end_inx);
                remaining_deps = atomic_fetch_sub_explicit(
                    &tl_current_task->unfinished_dependencies,
                    1,
                    memory_order_acq_rel
                );
                if (remaining_deps != 1) {
                    break; // if continuation spawned new children that
are still running , break and wait for them to finish
                }
            }
            if (remaining_deps == 1) {
                sc_task* parent = tl_current_task->parent_task;
```

```
                memory_pool_release(tl_current_task);
                // Traverse the DAG to resolve cascading unparks
                while (parent != NULL) {
                    size_t parent_dependencies =
atomic_fetch_sub_explicit(
                        &parent->unfinished_dependencies, 1,
memory_order_acq_rel
                    );
                    if (parent_dependencies != 1) {
                        // parent still has other unfinished children or
hasnt dropped its virtual lock yet
                        break;
                    }
                    // this is the last dependency of the parent
                    // check if it has a continuation function to run(same
process as before)
                    while (parent->task_fn != NULL) {
atomic_store_explicit(&parent->unfinished_dependencies, 1,
memory_order_relaxed);

                        if (chase_lev_push(&local_worker->cl_deque,
parent)) {
                            // Successfully scheduled parent's
continuation on our deque!
                            parent = NULL;
                            break;
                        }
                        // Deque is full: execute parent's continuation
inline
                        tl_current_task = parent;
                        sc_task_fn parent_fn = parent->task_fn;
                        parent->task_fn = NULL;
                        parent_fn(
                            parent->task_payload,
                            parent->start_inx,
                            parent->end_inx
                        );
                        size_t parent_rem = atomic_fetch_sub_explicit(
                            &parent->unfinished_dependencies, 1,
memory_order_acq_rel
```

```
                        );
                        if (parent_rem != 1) {
                            // Parent's continuation spawned new async
children, stop traversing the DAG
                            parent = NULL;
                            break;
                        }
                    }
                    if (parent == NULL) {
                        break;
                    }
                    // parent has no continuation left, so it is
completely done
                    // so recycle it and continue to the enxt parent
                    sc_task* next_parent = parent->parent_task;
                    memory_pool_release(parent);
                    parent = next_parent;
                }
            }
        }
    }
    return NULL;
}
```

After initializing, it starts the main loop which can be divided into steps:

1. Try to pop a task from the deque
2. If unsuccessful, try to pop a task from the spsc buffer
3. If also unsuccessful, try to steal a task from another worker
4. If a task was found, execute it
5. Decrement the current task remaining dependencies(dropping the virtual lock)
6. If there are no more dependencies, reset the virtual lock and execute the continuation function of the current task.
7. As long as the dependency counter becomes 0 after each function run and the continuation of the continuation function isn't null, keep executing the continuation.
8. Once there's no more continuation, decrement the dependencies of the parent of the current task.
9. While the parent dependencies remain 0 and the parent isn't NULL, continue to decrement and move to the parent task, while either pushing the parent's continuation task to the deque and break, or if unsuccessful, execute it inline and do the same process as 7-8 but for the parent and at the end continue to the next parent.

The main loop itself isnt that complex, all you need is to understand the parts that make it.

The job splitter

The splitter's purpose

The last thing we need to discuss before moving on to the scheduler's api, is the job splitter.
The internal macro job splitter is a function assigned to the first task allocated when submitting a job, instead of the function the developer gave.
Its only purpose is to recursively split the job into 2 parts, push the right task into the deque, and recursively split the left part again.
It does it until it reaches the chunk size provided by the developer, and only then it actually executes the developers function.
There are 3 main reasons for why we use the macro job splitter:

1. **Maximizing Work-Stealing Efficiency:** When a worker goes idle and becomes a thief, it steals from the top of a busy worker's deque. If we used a flat approach, every task would represent a tiny, final chunk of work. A thief would steal one micro-task, finish it in a microsecond, and immediately attempt to steal again, causing atomic contention. With the recursive splitter, the oldest task at the top of the deque represents a massive, undivided branch (e.g., 50% of the entire remaining job). The thief steals this massive chunk, takes it back to its own core, and recursively splits it on its own deque. The thief secures a huge amount of localized work in a single operation, keeping cross-core traffic significantly lower.
2. **Reducing dependencies per parent:** if we use the flat approach, and have for example 100 tasks all pointing to the same parent, the parent dependencies counter will constantly change, invalidate cache lines among workers, cause more atomic contention,and cause more overhead overall. By using the splitter approach, every task has at max 2 children, which would increase performance.
3. **Splitting task spawning work between workers:** If we use a flat for loop to spawn 10,000 tasks, one single CPU core is doing 100% of the initialization and pushing. While worker 1 spends valuable milliseconds looping, pushing, and incrementing atomic counters, workers 2, 3, 4,etc are waiting for work.
   With the recursive splitter, task generation itself becomes a multi-threaded operation. Worker 1 splits the job into two halves and pushes one. Worker 2 steals it, and now both cores are splitting data simultaneously. Task generation leverages all available cores just to create the workload.

The job splitter function implementation

```
void internal_macro_job_splitter(void* payload, size_t start_idx, size_t
end_idx) {
    sc_job* job = (sc_job*)payload;
    // root task handles the completion
    scheduler_set_continuation(no_op_continuation);
    while ((end_idx - start_idx) > job->chunk_size) {
        size_t total_elements = end_idx - start_idx;
        size_t mid_point = start_idx + (total_elements / 2);
        size_t remainder = mid_point % job->chunk_size;
        if (remainder != 0) {
            mid_point += (job->chunk_size - remainder);
        }
        if (mid_point >= end_idx) break;
        sc_task* right_task = memory_pool_acquire();
        if (!right_task) {
            job->job_function(job->job_payload, mid_point, end_idx);
            end_idx = mid_point;
            continue;
        }
        right_task->task_payload = payload;
        right_task->start_inx = mid_point;
        right_task->end_inx = end_idx;
        right_task->task_fn = internal_macro_job_splitter;
        right_task->parent_task = tl_current_task;
        //set up virtual lock
        atomic_store_explicit(&right_task->unfinished_dependencies, 1,
memory_order_relaxed);
      atomic_fetch_add_explicit(&tl_current_task->unfinished_dependencies,
1, memory_order_relaxed);
        if (!chase_lev_push(&tl_current_worker->cl_deque, right_task)) {
            //rollback
      atomic_fetch_sub_explicit(&tl_current_task->unfinished_dependencies,
1, memory_order_relaxed);
            memory_pool_release(right_task);
            job->job_function(job->job_payload, mid_point, end_idx);
        }

        end_idx = mid_point;
    }
```

```
    tl_current_task->task_payload = job->job_payload;
    job->job_function(job->job_payload, start_idx, end_idx);
}
```

The API functions

Here are all of the functions that belong to the api:

```
//sets up the workers, the spsc buffer for task transmissioning, and the
chase-lev deque(rounds queue sizes up to nearest power of 2).
//required before any form of job submission.
//after calling boot, when you dont want the threads to keep running, call
scheduler_stop_workers() to stop the threads and free all the memory.
void scheduler_boot(size_t spsc_queue_size,size_t cl_deque_size);

//pass a ptr to an pre allocated job struct,and initialize the struct
void scheduler_init_job(sc_job* empty_job,void* payload,size_t
payload_size,sc_job_fn job_fn);

//starts the job, while deciding the chunk division of the payload(which
willl be later split into tasks of that chunk size).
//any illegal size will result in the default 128 chunk size(above
payload_size). if you want the default chunk size, pass 0
bool scheduler_submit_job(sc_job* job,size_t chunk_size);

static inline bool scheduler_is_job_complete(sc_job* job) {
    return atomic_load(&job->is_done);
}
//spawns child tasks that the current task executing will be dependent on.
//will ONLY work when called from within the parent task function.
// choose wisely what the chunk size will be since it will be pushed in
256 task batch and any task that doesnt fit/fails to be allocated-
//will be executed inline
void scheduler_spawn_subtasks(void* payload, size_t start_idx, size_t
end_idx, size_t chunk_size, sc_job_fn task_fn);
//sets what the current task will do after all child sub tasks finish and
return back to the parent
void scheduler_set_continuation(sc_job_fn next_task);
//cleans all the allocated memory, and frees all the worker threads
void scheduler_stop_workers();

// blocks the calling thread until the job is complete
void scheduler_wait_for_job(sc_job* job);
```

Comparing performance- does it actually matter?

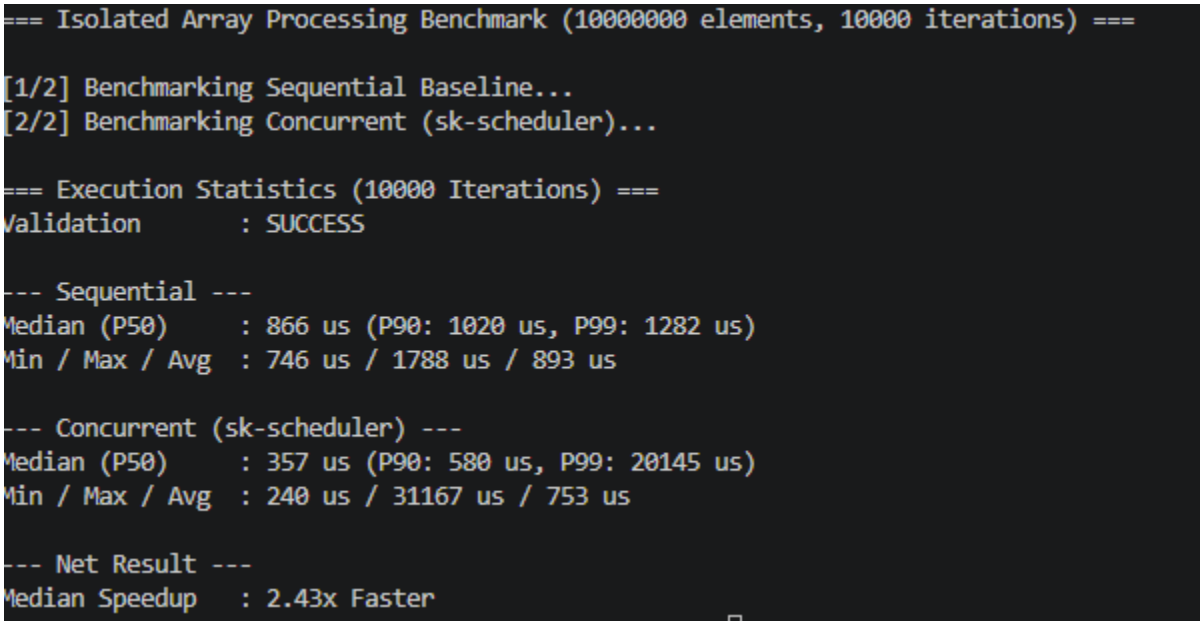
So after you know everything that goes on in the scheduler, let's see how it actually holds when compared to other methods in terms of performance(All of the source codes for all tests are available on my github inside the examples/ folder and were written using Gemini 4 Argon).
I have made 4 main comparisons:

1. Simple array updating- both using the scheduler and sequentially.
2. Merge sort- using a recursive approach and the job approach using spawn sub task
3. Quick sort- using a normal concurrent recursive approach and using the scheduler
4. Performing the calculations to render the Mandelbrot set on screen- comparing sequential, concurrent and the scheduler.

And the results were very interesting.

Simple array writing

This benchmark test was to compare the most basic type of use of the scheduler- writing simple integer data into the array.
In this benchmark test two approaches were compared- first is the use of a simple loop through all the elements sequentially, and the second was to split the array writing between the worker threads using the scheduler.
The reason for even doing this test is to see the scheduler at its worst- where the data needed to be processed is uniformed(meaning every chunk does the same amount of work), and the entire process is extremely fast, so the overhead caused by the scheduler itself will be the most prominent.
The results are as such for 10,000,00 elements and 10,000 iterations:



Looking at the results I am extremely impressed by the scheduler.i originally thought it would be absolutely destroyed by the sequential approach simply because of the overhead, but it seems that even though the sequential approach is most likely already more optimized by the compiler and the CPU than the scheduler approach.
It managed to prove how little the overhead is in most of the cases, performed better both by comparing the minimum and the median.
To address the 30ms elephant in the room, the max and avg results were unfortunately expected.
I haven't yet figured out what is the direct cause of this type of behavior, but considering the difference between P90 and P99 is THAT big, i concluded it may just be all down to luck- maybe how many other processes are running, how many hardware interrupts happen during it,maybe the main thread has a quirk I haven't yet found or understood. But only looking at the raw numbers, I can't tell for certain.
If you're wondering, the results for 1 million elements weren't that different, same for 100,000 elements which also makes me even more surprised.

Merge sort.

The second benchmark test was mainly to stress test the spawn subtask and to check how well does the scheduler fare against a recursive algorithm.
The test was comparing two merge sort algorithms- one is a normal text book merge sort(or more specifically, tim sort), and the second is a recursive-like merge sort using the scheduler where every recursive call is creating a sub task with half the size of the original task for both left and right, just like normal merge sort.
The base case is when the chunk is <=100 and it switches to insertion sort, since it was the most effective for both.
Here are the results for 1,000,000 elements and 100 iterations:

```
=== Starting Isolated Sort Benchmark (1000000 elements, 100 runs) ===

[1/2] Benchmarking Sequential Baseline...
[2/2] Benchmarking Concurrent (sk-scheduler)...

=== Final Execution Statistics (100 Runs) ===
Validation       : SUCCESS

--- Sequential ---
Median (P50)     : 46.23 ms
P90 / P99        : 49.90 ms / 52.68 ms
Min / Max        : 44.49 ms / 52.68 ms
Mean (Average)   : 46.95 ms

--- Concurrent (sk-scheduler) ---
Median (P50)     : 11.66 ms
P90 / P99        : 28.12 ms / 32.08 ms
Min / Max        : 10.82 ms / 32.08 ms
Mean (Average)   : 18.02 ms

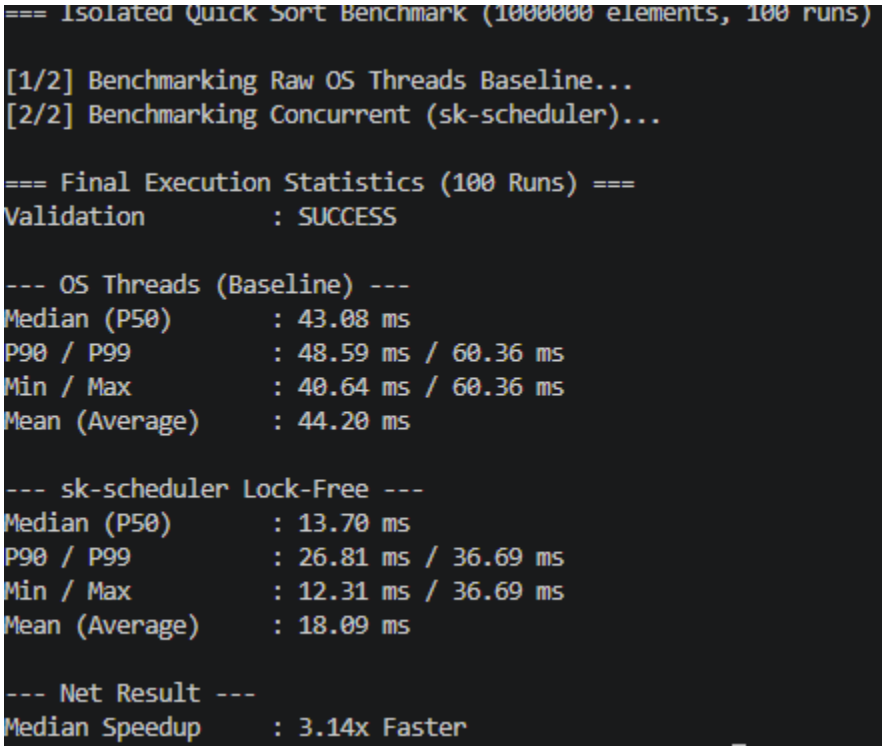
--- Net Result ---
Median Speedup   : 3.97x Faster
```

This result is incredibly good, while we can still see that same 30ms overhead I talked about before, everything else proves that even when comparing recursive-like structures, the scheduler still performs better. A 4x speedup is an incredibly impressive result.

The result for the same test but with 100,000 elements was a 3.37x speedup, which is similar enough to the first test, proving the scheduler efficiency even more.

Concurrent Quick sort

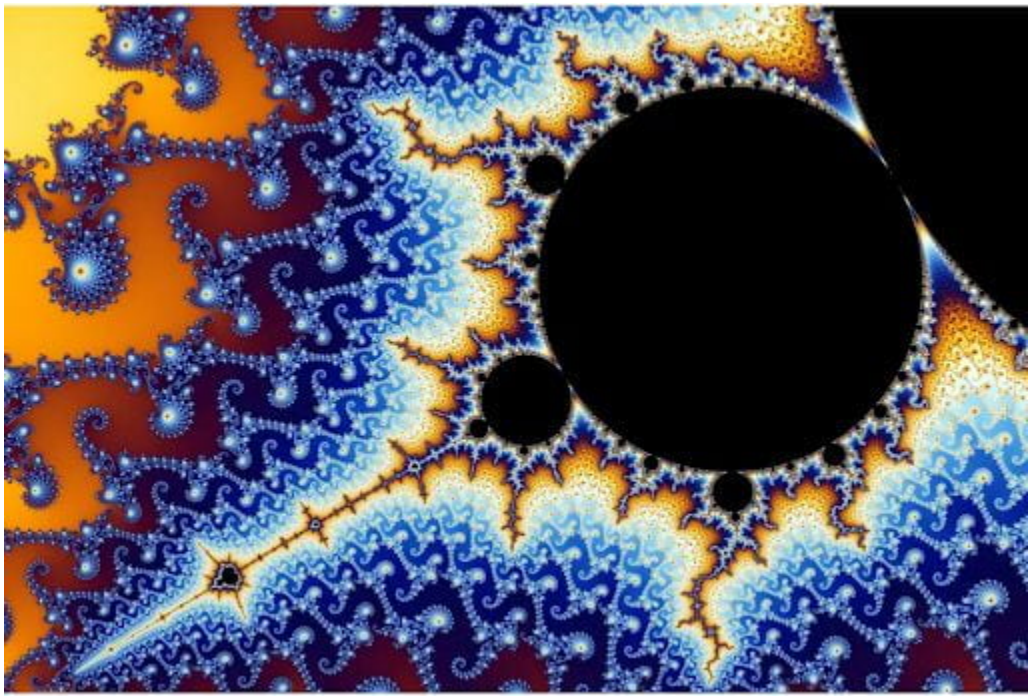
The third benchmark was to actually compare the two methods we compared only theoretically in the beginning- the first is a quick sort where every step a thread is created to encapsulate a recursive step inside its own thread, and the second is similar to the merge sort, but for the quick sort algorithm.
The quick sort algorithm is still very simple work done in each task by itself and also its relatively uniformed when the array is random.
Here are the results for 1,000,000 elements and 100 iterations:



We have proved that the overhead caused by creating os threads for each task is much higher than the overhead caused by the scheduler.
But remember to take this info with a grain of salt since creating threads is also a task that requires OS resources and also takes time,which our scheduler avoids during boot time while the raw OS threads does not and creates threads on the fly.

Mandelbrot set rendering

This is the most important test out of any other. This test differs from the rest because it works on un-uniformed data where each chunk isn't doing the same amount of work- perfect for work stealing.
For those who don't know, this is the mandelbrot set visualized:



Its a set of complex numbers that creates an infinitely detailed fractal shape when plotted on a graph.
I'll not get into the full details but all you need to know is that if we wanted to render this image by pixels, each pixel would be doing a different amount of work depending on where it is located.
So rendering this would be an un-uniformed algorithm.
In this test I compared ALL three approaches used so far- sequentially, using static threads each digesting the same percent from the image, and the third is with the scheduler.

Running the benchmark on a virtual `1920x1080` screen with `50` runs gives us this result :

```
=== 3-Way Mandelbrot Work-Stealing Benchmark (1920x1080, 1000 max_iter, 50 runs, 12 cores) ===

[1/3] Benchmarking Sequential Renderer (1 Core)...
  Sequential run 50 of 50...
[2/3] Benchmarking Static Thread Partitioning (12 Cores, Equal 1/12 Slices)...
  Static Threads run 50 of 50...
[3/3] Benchmarking sk-scheduler Work-Stealing Renderer (12 Cores)...
  sk-scheduler run 50 of 50...

=== Final Execution Statistics (50 Runs) ===
Validation                      : SUCCESS (Bit-exact 1920x1080 match)

--- 1. Sequential (1 Core) ---
Median (P50)                    : 1035.43 ms
Min / Max / Avg                 : 1004.97 ms / 1056.27 ms / 1034.93 ms

--- 2. Static Partitioning (12 Cores, No Stealing) ---
Median (P50)                    : 267.58 ms
Min / Max / Avg                 : 192.24 ms / 415.03 ms / 275.82 ms

--- 3. sk-scheduler Work-Stealing (12 Cores) ---
Median (P50)                    : 113.11 ms
Min / Max / Avg                 : 99.17 ms / 137.59 ms / 112.73 ms

--- Net Speedup Results ---
Static Threads vs Sequential    : 3.87x Faster
sk-scheduler vs Static Threads  : 2.37x Faster (Work-Stealing Gain!)
sk-scheduler vs Sequential      : 9.15x Faster
```

The result is exactly what we were looking for. This proves that when the data processed is un-uniformed, the scheduler makes such a huge impact on performance just by switching from static threads to dynamic task assignment using the scheduler's workers.
This is an absolute win for the scheduler, and you can imagine how much more impactful it could be when the amount of needed to be done by each task increases, like in game engine loops where you need to process every single entity in the game in real time with the least amount of overhead as possible and most efficiently.
It seems like we can assume that we have successfully built something that can at least be useful in some scenarios.

Conclusion

After exploring everything that the scheduler has to offer and seeing the impact the tool can make in some cases, we have reached the end.

The project took me around 3-4 weeks to build by myself, including researching, building prototypes, testing and bug hunting.

I have learned so much from this project, not just about C and system designing, but also how the hardware actually operates- from the CPU's themselves to the L1 caches.

I encourage you to build a project like this yourself, because for me it was incredibly eye-opening to learn so many new concepts and to be able to look at efficiency not as just run-time complexity, but to actually measure down to the last nanosecond and see where i can shave off as much overhead as possible.

Finally,as much as it is currently really a controversial time for different advanced LLMs such as Claude or ChatGPT,If you want to build a complex project to genuinely learn, I highly recommend using an AI like Gemini as a research partner. While his coding capabilities are pretty bad(this was written before Argon 4 btw) and in general you shouldn't rely on AI to just write the entire code for you, its deep research capabilities(and now also its bug hunting capabilities) makes it incredibly easy to find and summarize highly technical professional material while keeping you the actual architect and engineer of the project.

I hope you found this documentation useful!